

Poszukiwanie skarbu

Limit czasu: 8 s Limit pamięci: 256 MiB

Szukasz dawno zaginionego skarbu na ogromnym polu komórek o wymiarach $N \times N$, korzystając z magicznego kompasu. Łącznie na polu ukrytych jest K skrzyń ze skarbami, znajdujących się w różnych komórkach. Twój cel jest prosty: znaleźć je wszystkie!

Kiedy umieścisz kompas na jednej z komórek, znajdzie on najkrótszą ścieżkę do skrzyni ze skarbem, używając wyłącznie ruchów w lewo, do góry, w prawo i w dół (czyli do najbliższej skrzyni według odległości Manhattan). Następnie wskaże kierunek pierwszego kroku. Jeśli istnieje wiele poprawnych pierwszych ruchów zmniejszających dystans do najbliższej skrzyni (lub kilku takich skrzyń), kompas zwróci wszystkie te kierunki. Jeśli dana komórka zawiera skarb, kompas wskaże to zamiast kierunku.

Za każdym razem, gdy znajdziesz skrzynię ze skarbem, możesz opróżnić jej zawartość, ale nie możesz usunąć samej skrzyni — jest zbyt ciężka. Twój kompas nie wie, czy skrzynia jest pełna czy pusta. Będzie wskazywał drogę do najbliższej skrzyni, niezależnie od tego, czy jest pusta, czy pełna.

Spróbuj znaleźć położenie wszystkich skrzyń ze skarbami, używając jak najmniejszej liczby zapytań do kompasu.

Zadanie

To jest zadanie interaktywne. W każdym przypadku testowym (tj. przy każdym uruchomieniu programu) twój program będzie musiał rozwiązać kilka poszukiwań skarbu. Powinieneś komunikować się z systemem sprawdzającym przy użyciu biblioteki dostarczonej przez organizatorów. Biblioteka zawiera następujące deklaracje:

- **void** *NextHunt*(**int** & N , **int** & K) — wywołaj tę funkcję, aby rozpocząć kolejne poszukiwanie skarbu. Zwróci ona rozmiar planszy w parametrze N oraz liczbę skarbów w parametrze K . Jeśli w bieżącym uruchomieniu nie ma już więcej poszukiwań do rozwiązania, funkcja ustawi N i K na -1 ; w takim przypadku powinieneś zakończyć program kodem wyjścia 0 . Zauważ, że możesz wywołać tę funkcję przed odnalezieniem wszystkich skarbów w bieżącym poszukiwaniu, np. jeśli starasz się uzyskać tylko część punktów.
- **enum** { *TREASURE* = 0 , *DIR_RIGHT* = 1 , *DIR_UP* = 2 , *DIR_LEFT* = 4 , *DIR_DOWN* = 8 }; — są to stałe używane przez wartości zwracane przez funkcję *Query* (patrz poniżej).
- **int** *Query*(**int** x , **int** y) — jeśli komórka o współrzędnych (x, y) zawiera skarb, funkcja zwraca *TREASURE*. W przeciwnym razie zwraca sumę jednej lub więcej spośród stałych *DIR_RIGHT*, *DIR_UP*, *DIR_LEFT* i *DIR_DOWN*, wskazując, które ruchy zwraca kompas umieszczony w komórce (x, y) . Współrzędne x i y muszą być liczbami całkowitymi z zakresu od 0 do $N - 1$. (Uwaga: w tym zadaniu współrzędne y rosną od góry do dołu.)

Po tym, jak *NextHunt* zwróci $N = K = -1$, twój program nie może już wywoływać ani *NextHunt*, ani *Query*. Nie może także wywołać *Query* przed pierwszym wywołaniem

NextHunt. Jeśli program nie będzie przestrzegał tych ograniczeń, albo wykona więcej niż 1000 zapytań w trakcie pojedynczego poszukiwania skarbu, albo przekaże do *Query* wartości x i/lub y spoza dozwolonego zakresu, biblioteka zakończy działanie programu i zwróci werdykt run-time-error (RTE) dla bieżącego przypadku testowego.

Skarb uważa się za odnaleziony, jeśli przynajmniej raz wykonano zapytanie dotyczące komórki, która go zawiera. Jeśli program wywoła *NextHunt* przed odnalezieniem wszystkich skarbów w bieżącym poszukiwaniu, nie jest to traktowane jako błąd, ale wpłynie na wynik (więcej informacji o ocenianiu poniżej).

Aby uzyskać dostęp do biblioteki, program powinien dołączyć plik nagłówkowy `treasurehuntlib.h`:

```
#include "treasurehuntlib.h"
```

Plik nagłówkowy można pobrać tutaj: `treasurehuntlib.h`.

Aby pomóc w tworzeniu rozwiązania, dostępna jest prosta implementacja biblioteki: `treasurehuntlib-public.cpp`. Aby skompilować ją wraz z programem, wystarczy dodać jej nazwę jako parametr kompilatora, np.:

```
g++ foo.cpp treasurehuntlib-public.cpp
```

jeśli `foo.cpp` jest nazwą pliku zawierającego twoje rozwiązanie.

Na serwerze oceniającym zostanie użyta inna implementacja biblioteki, więc nie powinienś zakładać niczego na temat tego, jak dokładnie działa. Możesz jednak założyć, że nie zanieczyszcza globalnej przestrzeni nazw (global namespace) żadnymi deklaracjami poza wymienionymi powyżej (*NextHunt*, *Query* oraz pięcioma stałymi).

Twój kod nie może czytać ze standardowego wejścia ani pisać na standardowe wyjście, ponieważ będą one wykorzystywane przez naszą implementację biblioteki na serwerze oceniającym do komunikacji z resztą środowiska oceny.

Wejście

Ograniczenia

- $1 \leq N \leq 10^6$
- $1 \leq K \leq 3$
- W pojedynczym uruchomieniu programu będzie co najwyżej 100 000 poszukiwań skarbu.
- W pojedynczym poszukiwaniu skarbu można wykonać co najwyżej 1000 zapytań.
- System oceniający nie jest złośliwy.

Podzadania

- Podzadanie 1 (10 punktów) $K = 1$
- Podzadanie 2 (30 punktów) $K = 2$
- Podzadanie 3 (60 punktów) $K = 3$.

Ocenianie

Podzadanie może składać się z wielu przypadków testowych (wielu uruchomień programu), a każdy przypadek testowy może zawierać wiele poszukiwań skarbu. Na potrzeby punktacji wszystkie poszukiwania skarbu z danego podzadania są oceniane wspólnie, niezależnie od tego, jak zostały pierwotnie podzielone pomiędzy przypadki testowe. Dla i -tego poszukiwania skarbu oznaczmy rozmiar planszy przez $N_i \times N_i$, liczbę skarbów przez K_i , liczbę zapytań wykonanych przez program przez Q_i , a liczbę odnalezionych skarbów przez F_i . Oznaczmy ponadto przez S całkowitą liczbę punktów dostępnych za to podzadanie. Wówczas liczba punktów przyznana programowi za to podzadanie wynosi:

- Jeśli program zawsze znajdował wszystkie skarby (tj. jeśli $F_i = K_i$ dla wszystkich i), jego wynik zależy od $t_i = \frac{Q_i}{\lceil \log_2 N_i \rceil}$:

$$\frac{S}{2} + \frac{S}{2} \cdot \min_i f(t_i) \text{ punktów, gdzie } f(t_i) = \begin{cases} 1, & t_i \leq 11 \\ 1 - (t_i - 11)/9, & 11 \leq t_i \leq 20 \\ 0, & t \geq 20. \end{cases}$$

- Jeśli program nie zawsze znajdował wszystkie skarby (tj. istnieje takie i , że $F_i < K_i$), otrzymuje

$$\frac{S}{2} \cdot \min_i \frac{F_i}{K_i} \text{ punktów.}$$

Innymi słowy, połowę punktów otrzymujesz za odnalezienie wszystkich skarbów, a drugą połowę za odnalezienie ich przy użyciu jak najmniejszej liczby zapytań. Aby uzyskać maksymalny wynik, rozwiązanie powinno odnajdywać wszystkie skarby przy użyciu co najwyżej $11 \lceil \log_2 N_i \rceil$ zapytań. Między $11 \lceil \log_2 N_i \rceil$ a $20 \lceil \log_2 N_i \rceil$ zapytaniami wynik maleje liniowo; jeśli natomiast rozwiązanie wykonuje więcej niż $20 \lceil \log_2 N_i \rceil$ zapytań, otrzyma tylko pierwszą połowę punktów za odnalezienie wszystkich skarbów. (Symbole $\lceil \cdot \rceil$ oznaczają zaokrąglenie wartości $\log_2 N_i$ w górę do najbliższej liczby całkowitej.)

Jeśli powyższe wzory dają niecałkowitą liczbę punktów za podzadanie, zostanie ona zaokrąglona do najbliższej liczby całkowitej.

Jeśli program zakończy się błędem wykonania lub nie będzie przestrzegał opisanego wcześniej protokołu korzystania z biblioteki, otrzyma 0 punktów za całe podzadanie. Aby otrzymać częściowe punkty za odnalezienie tylko części skarbów, program musi zatem zakończyć poszukiwanie w poprawny sposób, wywołując *NextHunt*.

Przykład

Wywołanie	Wartość zwrócona
<code>NextHunt(N, K)</code>	$N = 4, K = 1$
<code>Query(2, 0)</code>	$DIR_DOWN + DIR_RIGHT = 8 + 1 = 9$
<code>Query(3, 1)</code>	$DIR_DOWN = 8$
<code>Query(3, 2)</code>	$TREASURE = 0$
<code>NextHunt(N, K)</code>	$N = -1, K = -1$